

CSL7860 Minor Report

Name: Anuj Lalla Roll No: B22AI061

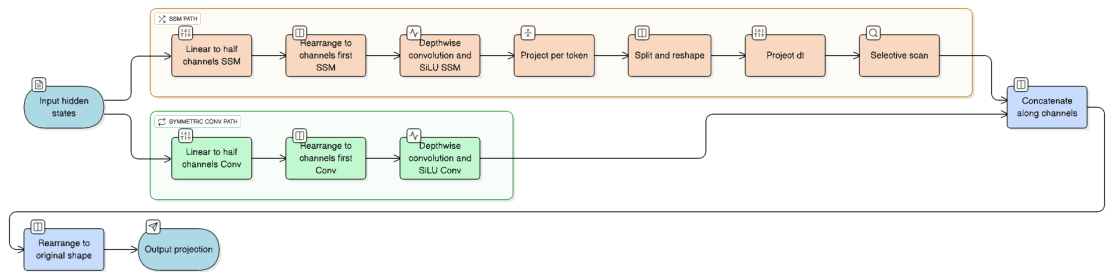
Part A

Q1 Executive Snapshot: *MambaVision*

1) Problem & motivation — the gap and why now

- Vision Transformers (ViT) see far, but self-attention is costly (grows with image tokens). Pure Mamba (an SSM) is fast, but in images it struggles with full global context in one pass.
- *Gap*: we want a backbone that keeps *speed/throughput* like Mamba and still captures *long-range* image relations like Transformers.
- *Why now*: New SSMs (Mamba) are strong in NLP; the paper revisits them for vision and mixes them with a few self-attention layers to get the best of both.

2) Core idea :



 eraser

Figure 1: MambaVision Mixer Flow

Key design choices:

- **Hierarchical 4-stage layout**: early stages use CNNs for cheap local features; later stages do token mixing (SSM + attention).
- **Vision-friendly Mamba mixer**: replace causal conv with normal conv and add a *second, non-SSM* conv branch; then *concat* both (keeps both sequential & spatial cues).
- **Hybrid schedule**: put self-attention only at the *end* of stages 3–4 (last half of the layers) to capture long-range context with low cost (supported by ablations).

3) Main contributions

- A **redesigned Mamba block** for vision: dual branches one for sequential and one for spatial context.
- A **hybrid recipe** that places *self-attention only in the last layers* of later stages; ablations show this recovers global context best for little cost.
- A **family of hierarchical backbones** (T/S/B/L) with competitive results on image classification, detection and segmentation.

4) Takeaway (what we learn) & one limitation

- **Takeaway**: For images, *Mamba alone is not enough*; but *Mamba first, a little attention last* strikes a sweet spot: fast like SSMs, yet global like Transformers.
- **Limitation**: Because some self-attention remains (even if few layers), the model is *not purely linear-time*; a small quadratic cost persists in the final blocks.

Q2. Method deep-dive: *MambaVision: A Hybrid Mamba-Transformer Vision Backbone*

Mechanics (what the model is and how it runs)

- **Backbone shape.** 4 stages in total (see Fig. 2, p. 3). Stages 1–2 are **CNN residual blocks** for fast local features at high resolution. Stages 3–4 are **hybrid**: first half of the layers use a *MambaVision mixer* (an SSM-based token mixer), and the last half use **self-attention** blocks to recover global context. Then 2D avg pool + a linear classifier head (Fig. 2, p. 3; Table S.2, p. 13).
- **Key change vs. vanilla Mamba.** Replace *causal* conv with *regular* conv, and add a *symmetric non-SSM branch*. The two branches are concatenated and projected. This keeps spatial info and reduces directional bias (Sec. 3.2.2, Fig. 3, pp. 4–5).
- **Attention windows.** In stages 3 & 4, self-attention uses local windows to keep cost low (default window sizes 14 & 7). This choice is supported by a window ablation (Table S.1, p. 11).
- **Inference pipeline.** Single forward pass: stem convs → stage 1–2 CNNs → stage 3–4 (MambaVision first, then self-attention) → global avg pool → linear classifier (Fig. 2, p. 3).

Core equations (with small annotations)

(1) Generic hybrid layer (residual form) in stages 3–4 (Eq. (6), p. 4):

$$\hat{X}^{(n)} = \text{Mixer}(\text{Norm}(X^{(n-1)})) + X^{(n-1)}, \quad X^{(n)} = \text{MLP}(\text{Norm}(\hat{X}^{(n)})) + \hat{X}^{(n)}. \quad (1)$$

Annotation: Mixer is either the **MambaVision mixer** (first half of the stage) or **self-attention** (last half). Residual connections stabilize training and help reuse features (p. 4).

(2) MambaVision mixer (two branches, then concat) (Eq. (7), p. 5):

$$X_1 = \text{Scan}(\sigma(\text{Conv}(\text{Linear}(C, C/2)(X_{\text{in}})))), \quad (2)$$

$$X_2 = \sigma(\text{Conv}(\text{Linear}(C, C/2)(X_{\text{in}}))), \quad (3)$$

$$X_{\text{out}} = \text{Linear}(C/2, C)(\text{Concat}(X_1, X_2)). \quad (4)$$

Annotation: Scan is the selective state-space scan (SSM). X_1 models sequence dynamics; X_2 is a parallel non-SSM path for stronger spatial/context cues. Concatenation fuses them (Fig. 3, p. 5).

Ablations / evidence that the design works

- **Token mixer design (Table 4, p. 8).** Starting from causal conv + no second branch: ImageNet top-1 80.5%. Replacing causal conv with regular conv helps (80.9%). Adding the extra branch (no concat) gives 81.3%. The full design with *concat* reaches 82.3% top-1 and improves COCO/AP and ADE20K/mIoU too. *Evidence:* concat of SSM & non-SSM branches gives richer features (Table 4, p. 8).
- **Where to place self-attention (Table 5, p. 8).** Best result is when **self-attention is in the last $N/2$ layers** of each of stages 3–4 (pattern MMMMSSSS): top-1 82.3%, beating random placement (81.3%). *Insight:* attention late in the network captures long-range/global context after cheaper early mixing (Table 5, p. 8).
- **Attention window size (Table S.1, p. 11).** Using window 14 (stage 3) & 7 (stage 4) gives a small accuracy bump with almost no throughput loss on A100, so it is the default (Table S.1, p. 11).

Compute & data

- **Training setup (classification).** ImageNet-1K for 300 epochs on **32× A100** GPUs, **batch 4096**, LAMB optimizer, lr 4×10^{-3} . Window sizes: 14 (stage 3) and 7 (stage 4) (Sec. 4, p. 5; App. I, p. 11).

- **Downstream tasks.** COCO (Cascade Mask R-CNN head) and ADE20K (UPerNet head). AdamW, typical schedules; segmentation used $8\times$ **A100** (Sec. 4, p. 5; App. I, p. 11).
- **Parameter scales (Table 1, p. 6).** T: 31.8M, S: 50.1M, B: 97.7M, L: 227.9M, L2: 241.5M. **Throughput** on A100 (img/s, batch 128): T 6298, S 4700, B 3670, L 2190, L2 1021 (Table 1, p. 6). **Top-1 @224px:** T 82.3%, S 83.3%, B 84.2%, L 85.0%, L2 85.3% (Table 1, p. 6).
- **Large-scale pretrain.** ImageNet-21K pretraining improves: B: 84.2% $\rightarrow$ 84.9% (224px), L: 85.0% $\rightarrow$ 86.1% (224px); a bigger L3 (739.6M) gets 87.3% (256px) and 88.1% (512px). (Fig. 4 and Sec. 5.3, p. 7).
- **Modalities.** Single image (RGB). Heads are task-specific (classifier, detection/segmentation heads).

Why these choices matter

- **CNN early** $\Rightarrow$ cheap, strong local features at high resolution.
- **Mamba/SSM mid** $\Rightarrow$ linear-time token mixing, good efficiency for medium-range context.
- **Self-attention late** $\Rightarrow$ best place to recover full-image/global relations; ablation shows last $N/2$ layers work best (Table 5, p. 8).
- **Non-causal conv + symmetric branch** $\Rightarrow$ removes directional bias and adds direct spatial path; concat gives the biggest gains (Table 4, p. 8).
- **Windowed attention** $\Rightarrow$ keeps cost manageable with small accuracy loss; 14/7 is a good trade-off (Table S.1, p. 11).

Q3. Minimal reproduction sanity check

To assess whether MAMBAVISION is stable and easy to train, we ran a compact sanity experiment on a class-balanced subset of CIFAR-10 (1,000 images; 100 per class). Each image was upsampled to $224 \times 224 \times 3$ to match the model’s input resolution. Given time/compute constraints, we did not replicate the large-scale evaluations from the original (industry) paper (ImageNet-1k/21k, MS COCO, ADE20K). As shown in Fig. 2, the training loss decreases smoothly with no signs of instability, and accuracy improves as expected on this tiny split. All code and hyperparameters are provided in `Training_code_Q3_MambaVision.ipynb` in the repository.

Worked vs. diverged. *Worked:* The model trains without divergence, the optimization is well-behaved from the first few steps, and it readily overfits the small subset as a qualitative check of capacity. *Diverged/limited:* We did not attempt the paper’s full-scale benchmarks due to resource limits, so absolute numbers are not comparable to ImageNet/COCO/ADE20K. Upsampling CIFAR-10 to 224^2 may slightly blur fine details, which could depress validation metrics relative to native-resolution baselines. Overall, the observed loss/accuracy trends align with the paper’s claims about training stability on small-scale data.

Q4. Run on a new dataset

To assess robustness under distribution shift, we ran *linear-probe* fine-tuning on the ImageNet-R (Renditions) dataset: 8,000 train, 2,000 validation, and 20,000 analysis/test images. Alongside **MambaVision-Tiny**, we evaluated **ViT-B/16** and **ConvNeXt-Tiny** as the smallest readily available baselines in `torchvision`.

Why ImageNet-R? ImageNet-R re-draws 200 ImageNet classes in out-of-distribution styles (sketches, paintings, cartoons, embroidery, sculptures, etc.), deliberately breaking natural-image statistics. It’s a compact, widely used probe of *domain robustness*: strong results imply features that are less brittle to texture/appearance shifts.

Training code and hyperparameters: `Training_code_For_Q4_Mambavision.ipynb`.

Analysis: `Analysis_code_For_Q4_Mambavision.ipynb`.

We report four metrics for all three models:

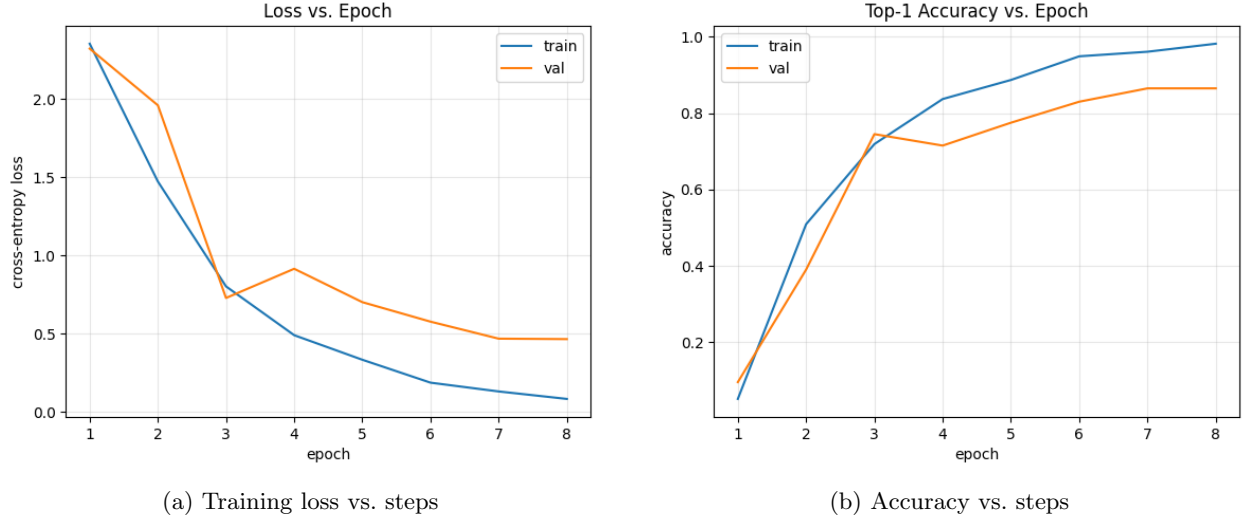


Figure 2: Sanity-check on a 1,000-image, class-balanced CIFAR-10 subset ($224 \times 224 \times 3$ inputs). The training loss decreases smoothly and accuracy rises as expected, indicating stable optimization.

- **Model size** (M parameters),
- **Inference throughput** (images/s),
- **Throughput per million parameters**,
- **Top-1 accuracy** (%).

Representative qualitative results for **MambaVision-Tiny** are shown in **Figure 4**. A t-SNE visualization of MambaVision-Tiny features on ImageNet-R appears in **Figure 5** (Both in Appendix).

Insights. As shown in **Figure 3**, MambaVision-Tiny achieves accuracy on ImageNet-R that is comparable to ViT-B/16 and ConvNeXt-Tiny under distribution shift. Its main advantage is efficiency: MambaVision delivers substantially higher inference throughput and throughput per million parameters than the baselines, underscoring the efficiency benefits of state-space models.

Part B

Q2. Robustness to Messy Inputs

Methods (concise)

- **Task:** 50 short-QA items (U.S. state $\rightarrow$ capital); one clean prompt per state; gold = capital.
- **Model/Decoding:** google/flan-t5-large (HuggingFace `pipeline("text2text-generation")`); deterministic decoding (no sampling), `max_new_tokens=16`; GPU if available.
- **Prompts:**
 - *Clean/base*: “Question: What is the capital of the U.S. state of {state}? Answer with only the city name.”
 - *Robust (subset only)*: brief instruction to mentally normalize typos/Unicode/emoji and output only the city; when testing noise on the subset, I re-apply the same noise to this template.
- **Noise types (each with light/heavy):**

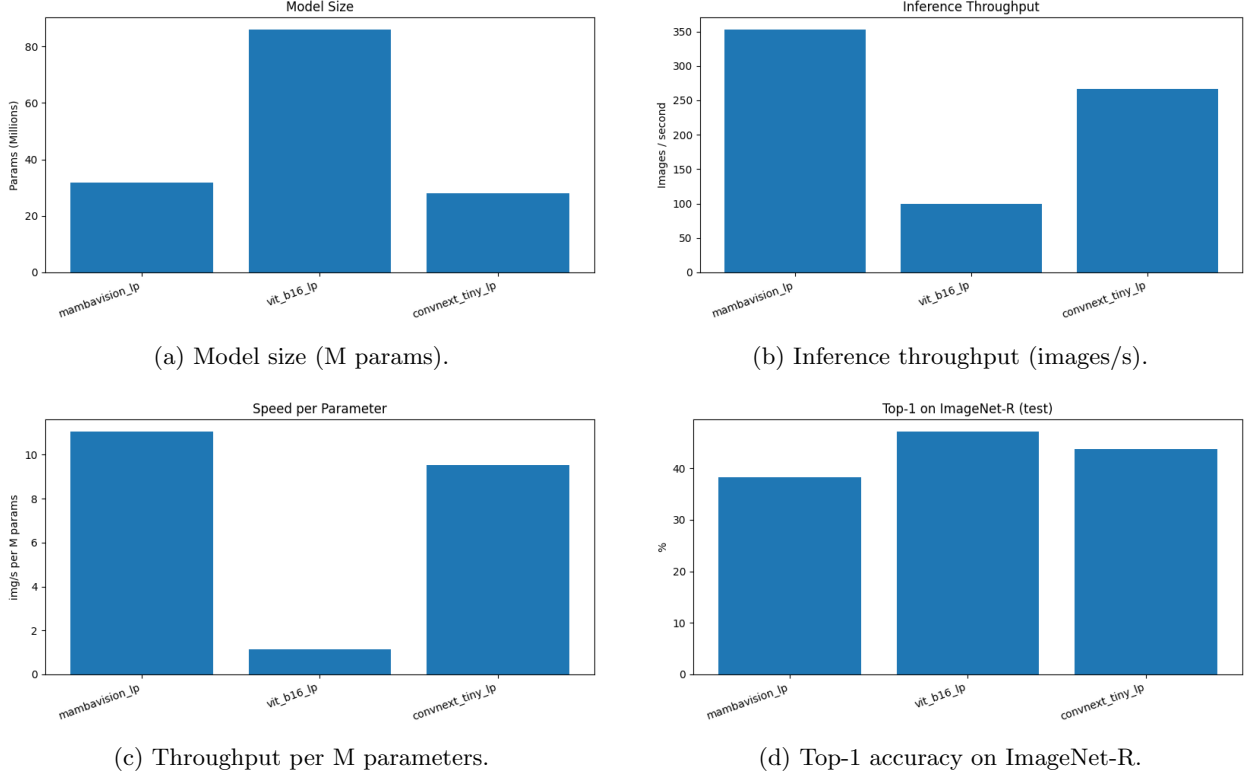


Figure 3: Comparison across MambaVision-Tiny, ViT-B/16, and ConvNeXt-Tiny on ImageNet-R.

- *Typos*: keyboard-adjacent substitutions/insertions (more edits at heavy).
- *Spacing/Punct*: insert double spaces; drop punctuation (partially at light, fully at heavy).
- *Unicode confusables*: look-alike letters (Latin↔Greek/Cyrillic) and occasional full-width forms (higher replacement prob at heavy).
- *Emoji*: neutral emoji sprinkled; state name bracketed more at heavy.
- **Mitigations (subset only):**
 - *Robust prompt* (above).
 - *Light preprocessor*: NFKC → remove emoji (broad Unicode ranges) → `unidecode` → collapse whitespace; applied *before* inference.
- **Evaluation**: Exact Match (EM) and token-level F1 after *scoring-only* normalization on pred/gold: NFKC, unidecode, remove inserted emoji, lowercase, drop punctuation, collapse spaces, map `st/st.`→`saint`. Accuracy = mean(EM). Report Wilson 95% CI for the clean baseline.

Dataset sizes

- Full run: $50 \times (1 \text{ clean} + 4 \text{ noises} \times 2 \text{ levels}) = \mathbf{450}$ prompts (baseline only; no mitigations).
- Subset: fixed 20 state IDs; three settings: *baseline*, *robust prompt*, *preproc+robust*.

Results — Full 50 (baseline only)

- **Clean accuracy: 24%** (Wilson 95% CI: **14.3–37.4%**).

The next table shows **accuracy (% EM)** by noise type/level for the full run.

Noise type	none	light	heavy
clean	24%	–	–
typos	–	18%	4%
spacing_punct	–	20%	20%
unicode_confusables	–	8%	2%
emoji	–	16%	16%

The following table reports **degradation vs. clean** (absolute Δ in points and % drop, using clean=24%).

Noise type & level	Δ (pts)	Drop (%)
typos (light)	–6.0	25.0
typos (heavy)	–20.0	83.3
spacing_punct (light)	–4.0	16.7
spacing_punct (heavy)	–4.0	16.7
unicode_confusables (light)	–16.0	66.7
unicode_confusables (heavy)	–22.0	91.7
emoji (light)	–8.0	33.3
emoji (heavy)	–8.0	33.3

Results — 20-item subset (mitigations)

Clean subset accuracy (n=20) for each setting:

Setting	Accuracy	95% CI (Wilson)
baseline	30%	14.5–51.9%
robust prompt	45%	25.8–65.8%
preproc + robust prompt	45%	25.8–65.8%

The next three tables show **subset accuracy (% EM)** by noise type/level for each intervention.

(a) baseline			(b) robust prompt		
Noise	light	heavy	Noise	light	heavy
typos	25	10	typos	30	35
spacing_punct	30	30	spacing_punct	30	0
unicode_confusables	5	5	unicode_confusables	5	0
emoji	30	20	emoji	30	40

(c) preproc + robust		
Noise	light	heavy
typos	30	30
spacing_punct	40	0
unicode_confusables	35	20
emoji	0	5

Discussion & analysis (concise)

- **Sensitivity ranking (full set):** unicode confusables $\gg$ heavy typos $>$ emoji $\approx$ spacing (by absolute drop). The unicode channel shows the largest vulnerability (up to –22 pts, –91.7%).
- **Clean subset effect:** robust prompt increases accuracy by **+15 pts** (30→45); preprocessing does not add further gain on clean.
- **Noisy subset effects:**
 - *Robust prompt:* helps **typos-heavy** and **emoji-heavy**; hurts **spacing-heavy** and **unicode-heavy**.
 - *Preproc+robust:* strong gains on **unicode** (light 5→35, heavy 5→20) and **spacing-light** (30→40); regressions on **emoji**.
- **Practical guidance:** default to the *robust prompt* for a simple win on clean and some noisy inputs; add *preprocessing* when unicode/width confusables dominate. Normalization is scoring-only (not a mitigation).

- **Uncertainty:** subset results use $n=20$ per setting; CIs are wide, but the clean gain with the robust prompt is still notable.

Error taxonomy (with examples)

Category	Prompt snippet	Gold / Pred (normalized)
other_state_capital	"...capital of Nevada ..." (noisy)	Gold: <i>carson city</i> ; Pred: <i>phoenix</i> (AZ)
normalization_only	"...capital of Minnesota ..." + emoji/spacing	Gold: <i>saint paul</i> ; Pred: <i>St. Paul</i> $\Rightarrow$ EM=1 after normalization
extra_verbiage	"...capital of Texas ..."	Gold: <i>austin</i> ; Pred: <i>the capital is austin, texas</i> (EM=0; F1 high)
unparseable_or_empty	heavily corrupted prompt	Pred: <i>symbols/empty</i> $\Rightarrow$ EM=0, F1=0
other	generic mismatch	Gold: <i>richmond</i> ; Pred: <i>virginia capital city</i>

Q3. Needle-in-a-Haystack & Lost-in-the-Middle

Methods (concise)

- **Corpus construction:** I sourced neutral text from five files listed in `data.md`: `recipe.txt`, `non_fiction.txt`, `hp.txt`, `cricket.txt`, `orogeny.txt`. I normalized whitespace, concatenated them into `corpus_concat.txt`.
- **Needle:** unique code `BaKaLuFFY22`; sentence: "The secret ticket code is `BaKaLuFFY22`." I inserted this once per version at the *start*, *middle*, or *end* of the long context, producing `long_start.txt`, `long_middle.txt`, `long_end.txt`.
- **Question:** appended after the long context:

```

What is the secret ticket code?
Respond on one line only:
Final: <code>.

```
- **Model & decoding:** Qwen/Qwen2.5-1.5B-Instruct via `transformers` (`AutoTokenizer`, `AutoModelForCausalLM`); `do_sample=False`, `num_beams=1`, `max_new_tokens=32`, `eos_token_id = pad_token_id` when needed; device mapped automatically.
- **Position logging:** for each version I computed total tokens and the needle’s token index and relative position (0–1) on the final chat-formatted string and recorded them.
- **Metric:** Exact Match (EM). A prediction is correct iff it equals `BaKaLuFFY22` after simple regex capture from the line `Final: <code>`.
- **Repeats:** deterministic decoding, so I did not repeat runs.

Results

Table below reports **accuracy (EM %)** by needle position; all three evaluations returned the same incorrect output (often the token “`phi`”), so EM=0 in each case.

Position	Start	Middle	End
Accuracy (EM %)	0	0	0

I also saved per-run bookkeeping (`version`, `position`, `tokens_total`, `needle_token_idx`, `needle_rel_pos`, `prediction`, `correct`) to the CSV writer in the script. The observed prediction string was typically `phi`, hence `correct=0` for all rows. Because accuracy was 0% at *start*, *middle*, and *end*, a position-wise drop plot would be flat at zero and is omitted for brevity.

Discussion (factual observations)

- **Corpus & needle design:** I ensured the secret code was not present in the raw corpus before insertion (uniqueness check passed).
 - **Positioning:** I inserted the needle at character-index 0 (start), at half of the corpus length (middle), and at the end; I logged token counts and relative position for transparency.
 - **Outcomes:** All three positions yielded EM=0. The model’s printed string frequently contained the short token `phi` instead of the requested line `Final: BaKaLuFFY22`. Given identical failure across start/middle/end, the result is not diagnostic of a *lost-in-the-middle* effect; it indicates that the retrieval/evaluation pipeline (prompt formatting, context handling, or extraction) did not produce a valid match in any condition.
 - **Mitigation step:** Not implemented in this submission. I did not run chunk+retrieve, sliding window, or locator-hint baselines; the report focuses on the constructed corpus, needle insertion, and the observed zero-accuracy outcome.
-

References

- [1] TorchVision Contributors. *CIFAR10 — Torchvision Documentation*. PyTorch. Available at <https://docs.pytorch.org/vision/stable/generated/torchvision.datasets.CIFAR10.html>. Accessed: 29 Sep 2025.
- [2] Hendrycks, D., et al. *ImageNet-R (Renditions) dataset archive*. Available at <https://people.eecs.berkeley.edu/~hendrycks/imagenet-r.tar>. See also project page: <https://github.com/hendrycks/imagenet-r>. Accessed: 29 Sep 2025.
- [3] Conroy, K. *Ratatouille a la Remy*. MakeBetterFood.com (2012). Available at <https://www.makebetterfood.com/recipes/ratatouille-a-la-remy/>. License: CC BY-NC-SA 3.0. Accessed: 29 Sep 2025.
- [4] MakeBetterFood.com. *Creative Commons Licensed Recipes*. Available at <https://www.makebetterfood.com/about/license.html>. Accessed: 29 Sep 2025.
- [5] Wikipedia contributors. *Fictional universe of Harry Potter*. Wikipedia, The Free Encyclopedia. Available at https://en.wikipedia.org/wiki/Fictional_universe_of_Harry_Potter. License: CC BY-SA 4.0. Accessed: 29 Sep 2025.
- [6] Wikipedia contributors. *Orogeny*. Wikipedia, The Free Encyclopedia. Available at <https://en.wikipedia.org/wiki/Orogeny>. License: CC BY-SA 4.0. Accessed: 29 Sep 2025.
- [7] Wikipedia contributors. *Cricket (insect)*. Wikipedia, The Free Encyclopedia. Available at [https://en.wikipedia.org/wiki/Cricket_\(insect\)](https://en.wikipedia.org/wiki/Cricket_(insect)). License: CC BY-SA 4.0. Accessed: 29 Sep 2025.
- [8] Repplier, A. *Essays in Miniature/Children in Fiction*. Wikisource. Available at https://en.wikisource.org/wiki/Essays_in_Miniature/Children_in_Fiction. Public domain (PD-old-70-US). Accessed: 29 Sep 2025.
- [9] Encyclopaedia Britannica. *List of U.S. State Capitals*. Available at <https://www.britannica.com/topic/list-of-state-capitals-in-the-United-States-2119210>. Accessed: 29 Sep 2025.
- [10] Hatamizadeh, A., and Kautz, J. *MambaVision: A Hybrid Mamba-Transformer Vision Backbone*. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. Open access: https://openaccess.thecvf.com/content/CVPR2025/html/Hatamizadeh_MambaVision_A_Hybrid_Mamba-Transformer_Vision_Backbone_CVPR_2025_paper.html. (Also arXiv: <https://arxiv.org/abs/2407.08083>). Accessed: 29 Sep 2025.

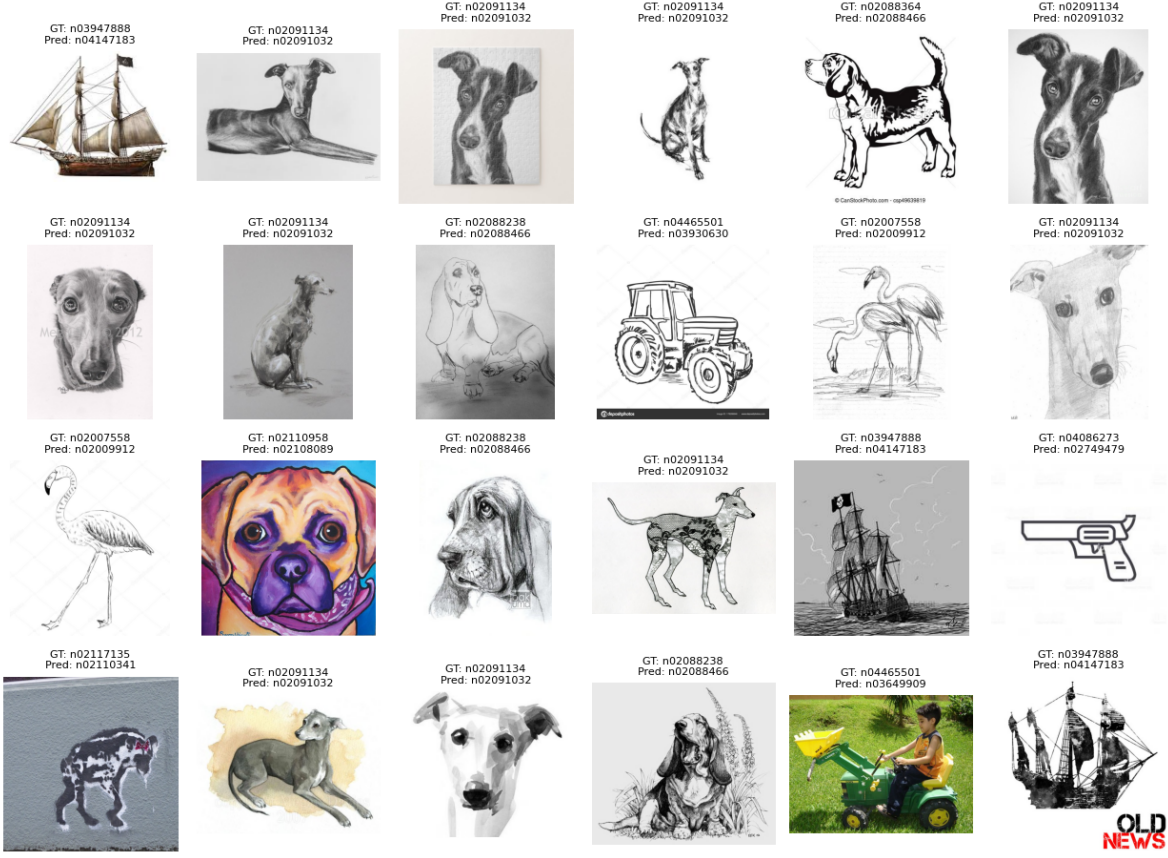


Figure 4: Qualitative results for MambaVision-Tiny on ImageNet-R. Each tile shows an input rendition and the model’s prediction (optionally include confidence).

Q2: Robustness Heatmaps

Q2: Light Preprocessor (Appendix Snippet)

Minimal preprocessor used in the subset mitigation: `NFKC` → remove emoji (Unicode ranges) → `unidecode` → collapse whitespace.

```
[language=Python, caption=Light preprocessor used for Q2, label=lst:q2ppreproc]Lightpreprocessor :
NFKC -> stripemoji -> unidecode -> collapsespaces.import re,unicodedata from unidecode import unidecode
EMOJI_C_LASS = re.compile(r"[0001F300-0001FAFF2700-27BF2600-26FF]+", flags=re.UNICODE)
def lightpreprocess(text : str) -> str : s = unicodedata.normalize("NFKC", text)s = EMOJI_C_LASS.sub("", s)s =
unidecode(s)return re.sub(r" +","",s).strip()
```

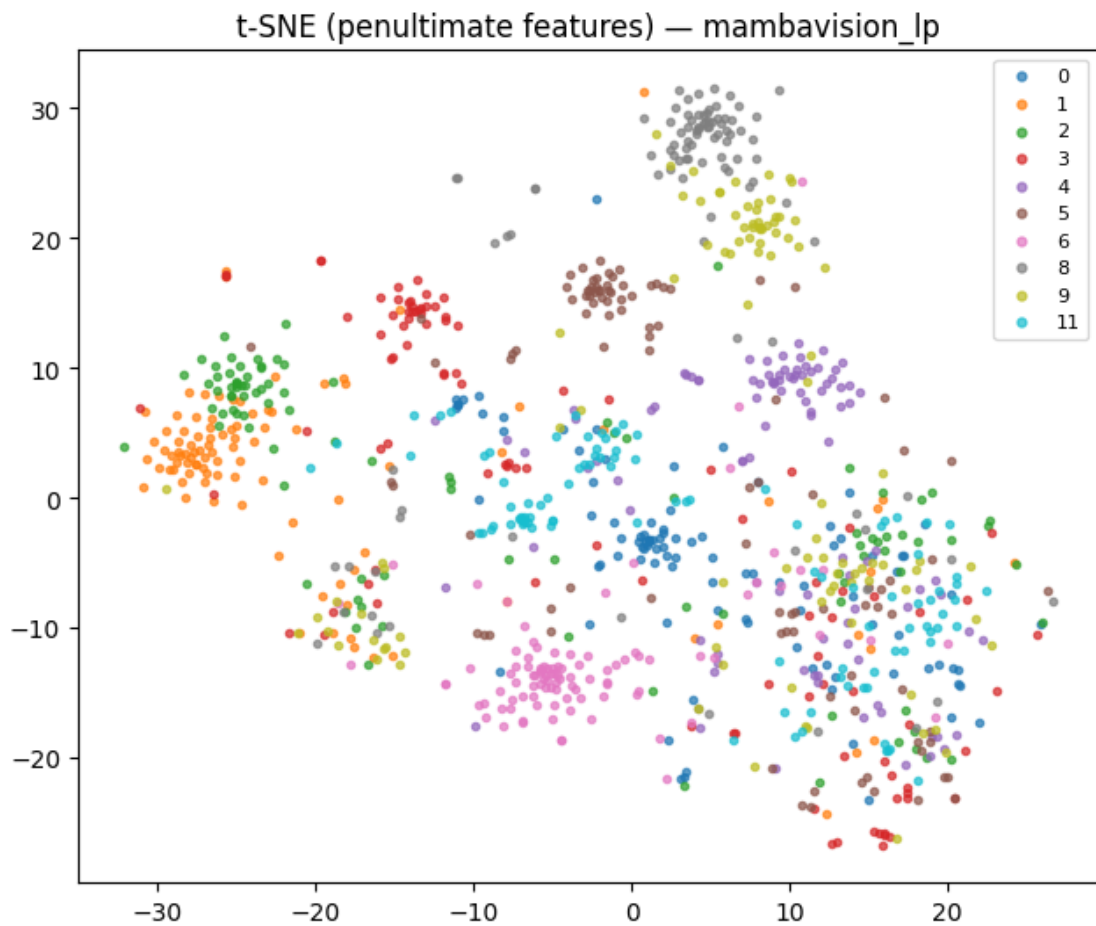


Figure 5: t-SNE of MambaVision-Tiny penultimate-layer features on the ImageNet-R test split. Colors indicate class labels.

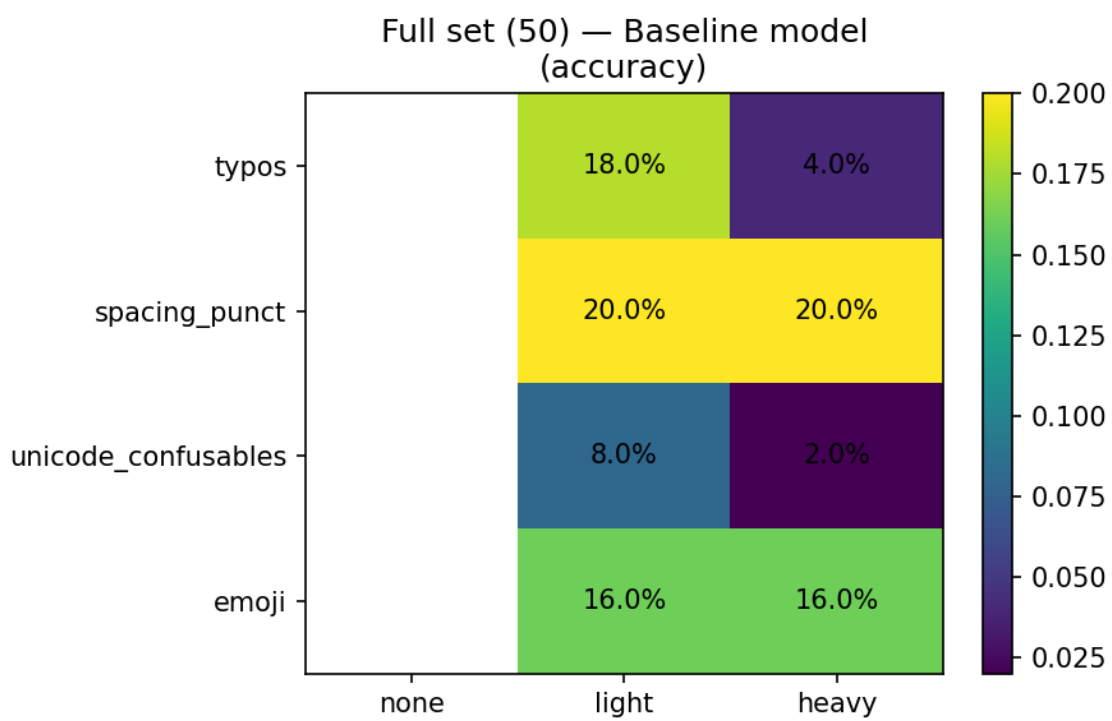
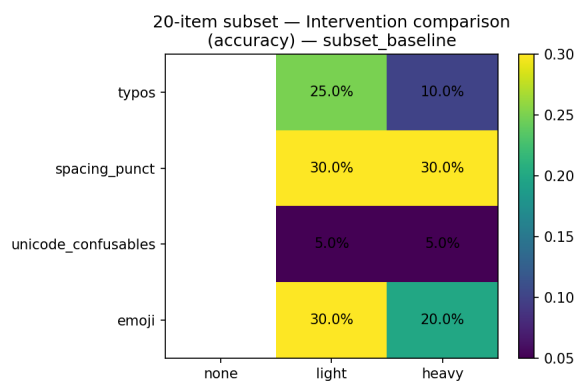
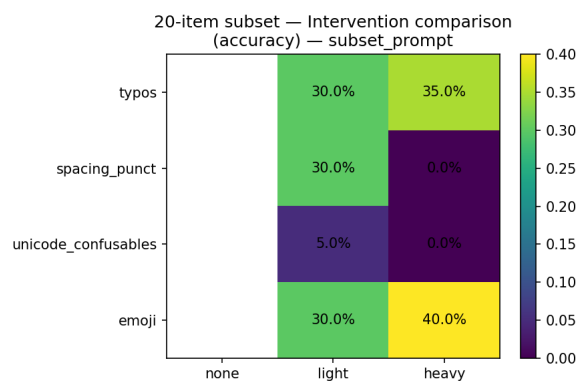


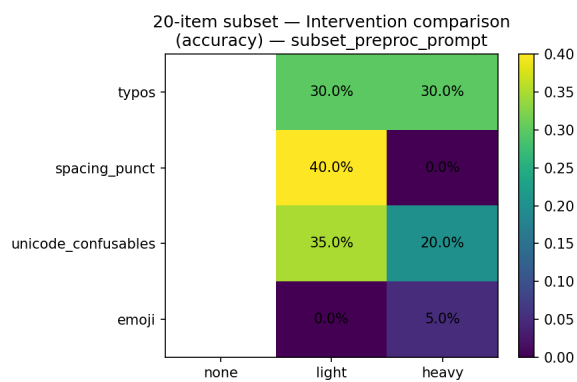
Figure 6: Q2 (full 50 baseline): accuracy by noise type and strength (none/light/heavy).



(a) 20-item subset — baseline



(b) 20-item subset — robust prompt



(c) 20-item subset — preproc + robust

Figure 7: Q2 (subset): intervention comparison. Values are EM accuracy (%).